四人 Git 协作规范

(A、B、C、D 对应团队的四名成员)

1 基本规则和分工

main 统一通过 PR 更新，由 A 合并。四人都在自己的任务分支提交和上传代码，不直接向 main 提交或推送，不强推公共历史。A 自己修改代码也创建 PR，由 A 自行合并。

一个任务一个分支，谁认领任务，谁在自己的分支开发。先 Fetch，再从 origin/main 创建；合并后删除，下个任务重新开分支。

一个 PR 一个目标，在交由 A 合并入 main 前，自己先确认代码能够构建，并说明本次改动和影响，A 核对目标分支、改动说明、构建结果及冲突状态后合并。

成员	分别做什么
A	与其他三人一样开发、提交和同步代码，另外负责统一合并 PR、在群中发布版本更新通知，以及协调冲突和比赛版本。
B / C / D	各自认领任务，在自己的分支开发和测试、发 PR、按意见修改、合并后清理分支。收到版本更新通知后完成同步。

谁修改代码，谁用自己的身份 Commit、Push 并创建 PR。A 合并其他人的 PR，不代替其他人复制代码后重新提交。

A 暂时无法处理时，由 A 明确指定 B/C/D 中一人临时完成合并和通知。

2 日常流程

每次开发

1 认领任务：开工前在群里说明自己准备修改的内容，避免重复开发。涉及其他人正在修改的部分，先商量。

2 保存当前工作：在当前任务分支处理未提交改动，工作区干净后再切分支。未完成工作用 Stash，新建文件也要保存时选 Stash (Include Untracked)；

3 创建分支：Fetch → Ctrl+Shift+P → Git: Create Branch From... → 选择 origin/main → 输入分支名。Fetch 不会自动更新本地 main 或当前分支，直接从更新后的 origin/main 创建即可。分支名用 feature-<Github ID>-<任务> 或 fix-<Github ID>-<任务>；

4 开发提交：检查当前分支和 diff，暂存本任务文件，按第 3 节写提交信息后 Commit。首次上传选 Publish Branch，远程选 origin；以后 Push。

5 发 PR：完成任务后，先同步最新 origin/main，确认代码能够构建，再 Push 并创建 PR，核对 base 为 main、compare 为自己的任务分支，说明改动内容和影响范围，并通知 A。

6 合并和同步：A 合并后立即在群里通知：谁修改了什么、影响范围、PR 链接和最新 main 的提交 SHA。其他三人看到通知后，可以回复“已同步 main 版本 ×××”；暂时没有同步的，回复“已知悉，暂未同步”，不要把收到通知或完成 Fetch 当作已经同步。
提交该 PR 的成员确认改动已合入 main 后，切离原任务分支，再清理本地和远程任务分支。

3 提交信息和 PR 描述

提交规范

<type>(<模块>): <简短中文描述>

普通提交和 PR 标题使用同一格式，模块必填；
Squash 标题沿用合规的 PR 标题。 feat 新功能、fix 修复、perf 性能或调参优化、refactor 重构、docs 文档注释、build 构建配置、chore 其他维护。

示例：
feat(路线): 新增八球路线第二段
fix(底盘): 修复 Turn 超时后 PID 状态未复位
build(仓库): 增加 clang-format 配置

一次提交只做一件事，不混入无关重构。普通代码提交前应能构建；阶段性工作可以 Commit、Push 到自己的任务分支，提交说明写清当前进度；准备合入 main 时必须能够构建。工作结束前，将需要保留和共享的改动上传，避免只留在本机。

调参探索和定稿

在自己的任务分支反复调参和上车试验，不必每次试验都提交或填写完整测试记录；有用的中间版本可 Commit 保存。参数定稿后整理成一个 PR，记录最终参数、测试条件和结果，再交由 A 合并。探索阶段的临时状态不能合入 main。

PR 描述模板

用具体事实替换括号内说明；不适用项写明原因

```
## 改了什么和为什么
- 改动和原因（说明任务及解决的问题）
## 影响范围
- 模块和调用方（列出受影响部分）
- 接口、端口、接线或依赖变化（无则写无）
## 验证记录
- 构建结果是否通过
- 上车：车型 / 配置 / 路线 / 结果（或不适用原因）
- 未验证部分及已知问题 （无则写无）
```

参数测试示例：
15in 机器人，八球路线第二段，终点偏差由约 40 mm 降至均小于 20 mm。

4 同步冲突和文件规则

同步 main 和解决冲突

在自己的任务分支、干净工作区中 Fetch，运行 Git: Merge...（部分版本为 Merge Branch...），选择 origin/main。不要切 main 直接合入任务分支。无冲突时重新构建和验证受影响功能，再 Push。
有冲突时，在 Merge Changes 打开文件，比较双方意图：Current 是当前任务分支，Incoming 是合入的 origin/main。Accept Both 只保留文本，不保证逻辑正确。手工整理代码，清除冲突标记，检查接口和参数含义。
谁的任务分支，谁解决其中的冲突。暂存解决后的文件，构建测试后完成 merge 提交、Push，更新 PR，并告知 A 冲突已处理，涉及他人改动时，一并说明处理结果
需要放弃合并时，先确认无新增工作需要保留，再执行 git merge --abort；失败后不继续 reset 或 Discard，先保存工作并找 A 协助。

提交和忽略哪些文件

文件	规则
产物和缓存	bin/、build/、compile_commands.json、固件 .bin、临时日志、__pycache__/ 不提交。
.vscode/	只共享 c_cpp_properties.json、settings.json、vex_project_settings.json；内容不含个人绝对路径、偏好或本机专属配置。
CSV 和资源	临时采集日志忽略；路线、标定、验收数据和必需资源可提交，说明用途与来源。
共享规则	.gitignore、.gitattributes、.clang-format 和 README 纳入仓库，通过 PR 修改。

5 回滚和比赛版本

main 出问题时

回滚仍走分支和 PR：从最新 origin/main 创建 fix 分支，确认需要撤销的是合入 main 后生成的 Squash 提交，执行下面的命令。完成构建和必要验证后，再 Commit、Push 并创建 PR。标题使用 fix(回滚): 撤销对应改动，正文记录原提交 SHA 和撤销原因，由 A 合并后通知全员。

保存可恢复的比赛版本

A 组织候选 main 完整构建并上车验证；A 不在按第 1 节处理。在 README 记录 tag、实际验证的提交 SHA、哪位成员测试及日期、车号、端口接线配置、工具链及部署步骤、测试项目和结果。创建带说明的 tag 并推送，确认 GitHub 可见。tag 必须指向实际验证过的提交，正式测试及比赛版本的实际代码应已保存为提交。不包含未记录的本地改动不移动或覆盖已发布的比赛 tag，修复后用新版本号。

赛前冻结和现场恢复

比赛前 24 小时冻结 main，不合新功能、重构或普通调参。必要 fix 由 A 确认修复范围，完成必要验证后合并，并通知全员同步。

现场故障优先恢复已验证版本：保存当前工作 → Fetch → Git: Checkout to... 选择比赛 tag → 使用匹配工具链构建 → 部署对应车辆 → 核对车号、配置和关键功能 → 通知使用的 tag。A 与熟悉部署的队友一起处理；A 不在按第 1 节处理。